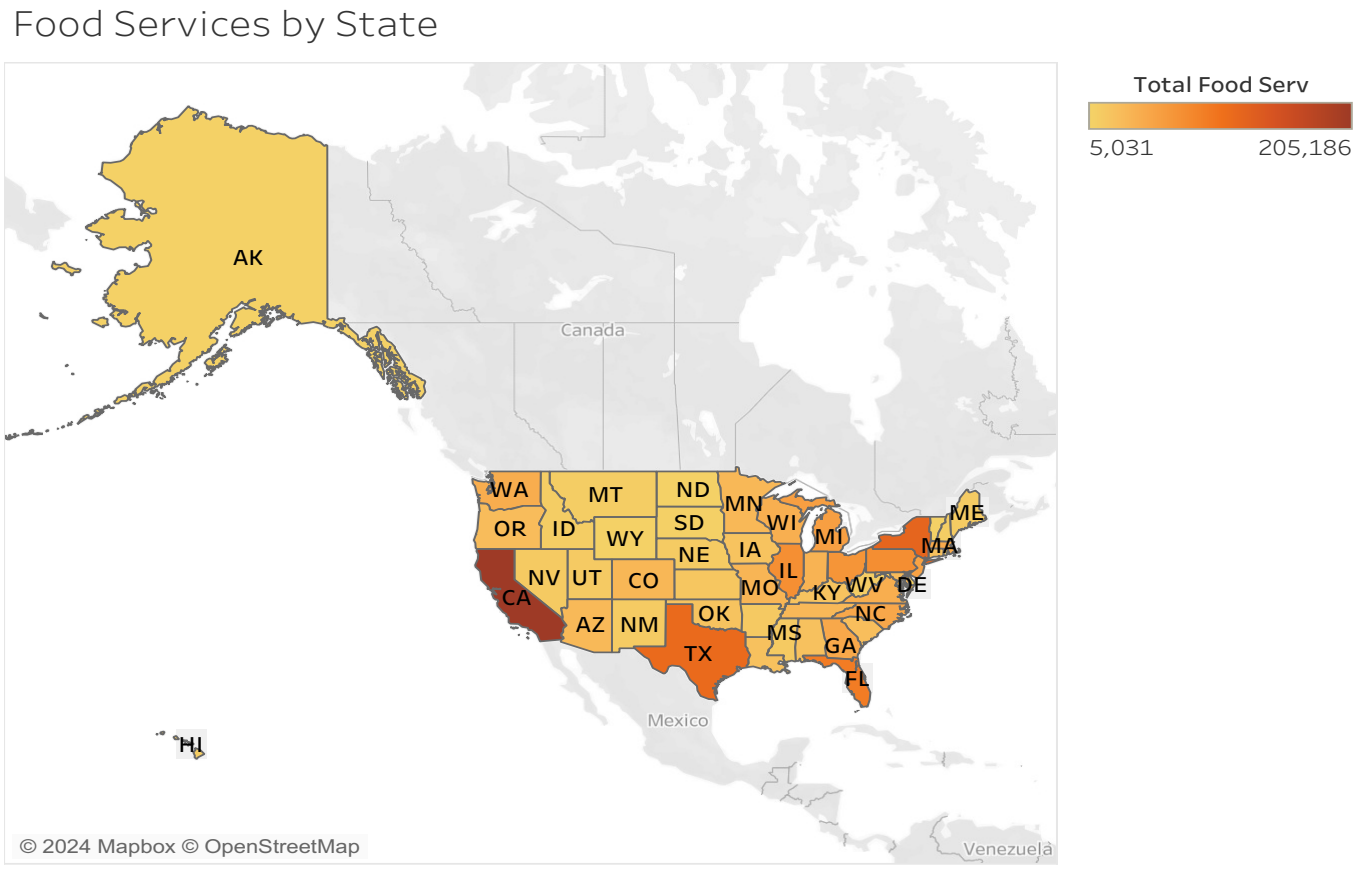
DSC 465- HOMEWORK 2

Name: Sanchal Sunil Dhurve

1) Download the FoodSrvcByCounty.txt file and create the following visualizations for this geographical data. The data is for the availability of food services by county in the U.S. It also has data by state (in the county field, some of them have the state names, and those rows hold the state totals, or you can aggregate by state)

a. Graph food services by state with an appropriate geographic visualization. Note any patterns that arise. Your visualization should clearly display states that have high levels or low levels of food service availability, so think carefully about the color scheme.

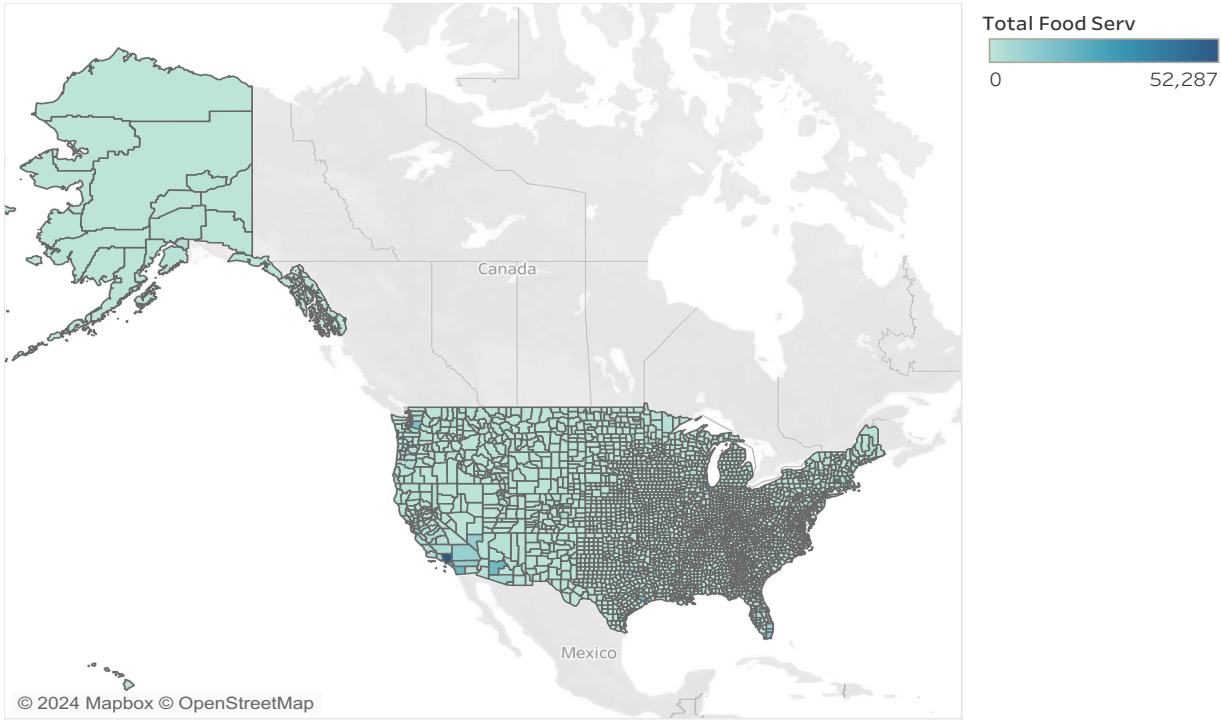


Map based on Longitude (generated) and Latitude (generated). Color shows sum of Total Food Serv. The marks are labeled by State. Details are shown for State. The view is filtered on State, which excludes Null.

The graph above illustrates variations in food service availability across states in the US.
By making use of the color divergence techniques (Orange-gold), it is easier identify the states which have higher and the ones which are lower in the level of service.
By making use of this technique, the lighter the orange shade, the lower the level of service, while the darker the orange shade, the higher the level of service.
California leads in food service availability, followed by New York, Texas and Florida.

b. Graph food services by county with the same type of visualization

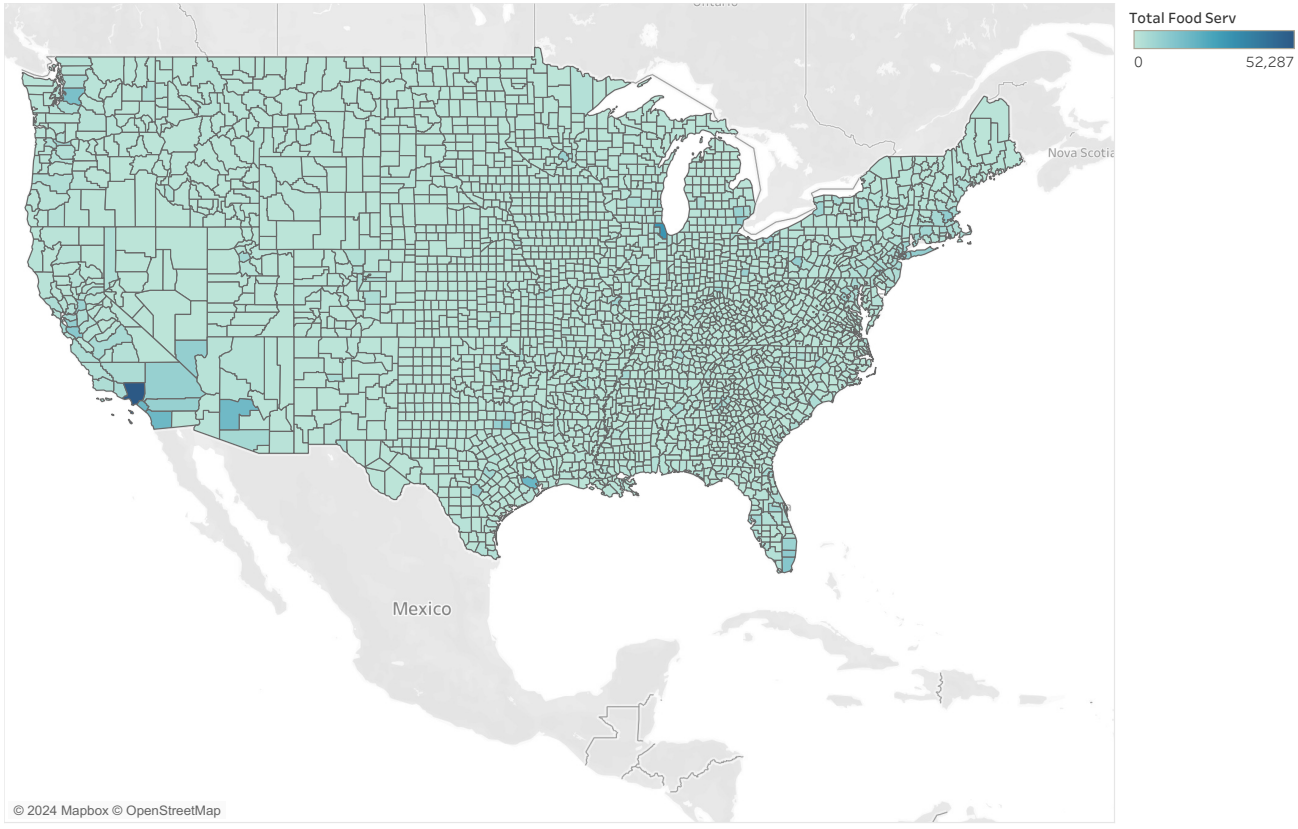
Food Services by County



Map based on Longitude (generated) and Latitude (generated). Color shows sum of Total Food Serv. Details are shown for State and County. The view is filtered on State, which excludes Null.

Zoomed in visualization

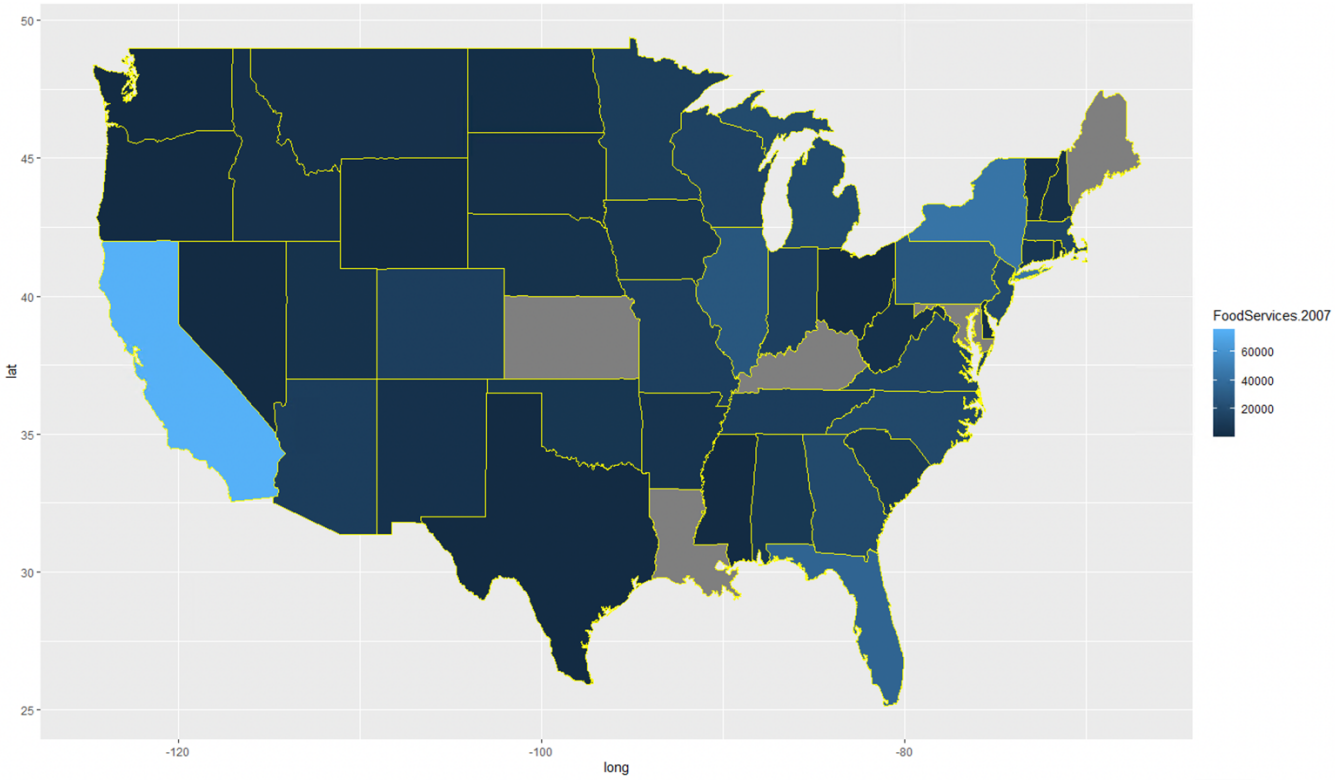
Food Services by County



Map based on Longitude (generated) and Latitude (generated). Color shows sum of Total Food Serv. Details are shown for State and County. The view is filtered on State, which excludes Null.

The graph above illustrates variations of food service availability by counties across the US. Food services are widely available in California's Los Angeles (highest), followed by and San Diego, Illinois' Cook County, and Texas' Harris and Dallas counties.

c. (Extra credit) Research how to do a diffusion or tile cartogram in R or D3 and create a cartogram of the state data from this dataset.



R codes-

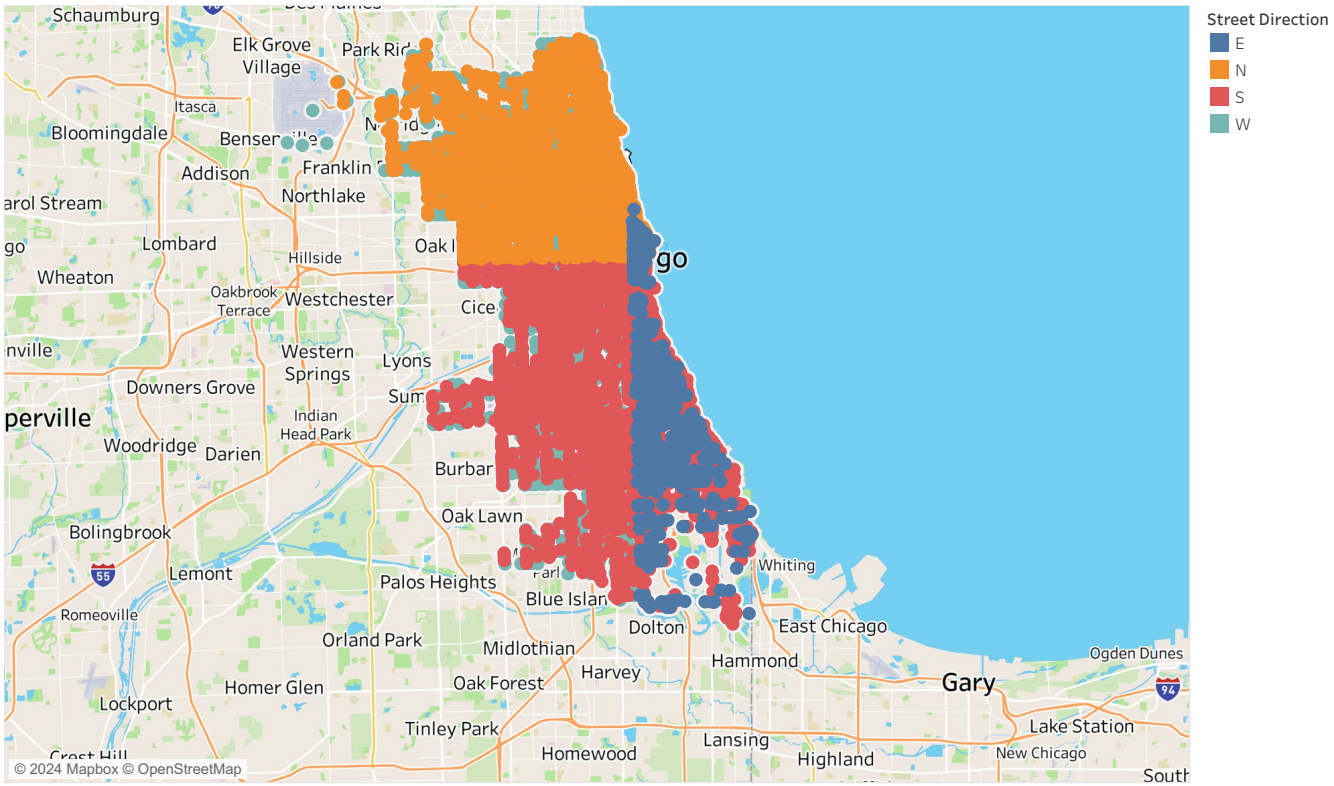
```
> rm(list = ls())
> library(usmap)
> library(ggplot2)
> library(dplyr)

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union

> food_data <- read.table("C:/Users/SDHURVE/Downloads/New folder/FoodSrvcByCounty.txt", header = TRUE, sep =
"\t", stringsAsFactors = TRUE)
> head(food_data)
         County State FoodServices.97 FoodServices.2002 FoodServices.2007
1 UNITED STATES               545068             565590             634361
2       ALABAMA                 6955               7075               8093
3       Autauga    AL             50                 67                 92
4       Baldwin    AL            303                319                414
5       Barbour    AL             42                 44                 45
6          Bibb    AL             12                 17                 19
> summary(food_data)
        County          State      FoodServices.97    FoodServices.2002  FoodServices.2007
 Washington:  26    TX     : 254   Min.   :     0.0   Min.   :     0.0   Min.   :     0.0
 Jefferson :  23    GA     : 159   1st Qu.:    21.0   1st Qu.:    20.0   1st Qu.:    21.0
 Franklin  :  22    VA     : 136   Median :    50.0   Median :    50.0   Median :    56.0
 Jackson   :  21    MO     : 115   Mean   :   552.8   Mean   :   573.9   Mean   :   644.3
 Lincoln   :  20    IL     : 102   3rd Qu.:   139.0   3rd Qu.:   145.0   3rd Qu.:   159.0
 Madison   :  18    NC     : 100   Max.   :545068.0   Max.   :565590.0   Max.   :634361.0
 (Other)   :2713   (Other):1977
> states <- map_data('state')
> head(states)
       long      lat group order  region subregion
1 -87.46201 30.38968     1     1 alabama      <NA>
2 -87.48493 30.37249     1     2 alabama      <NA>
3 -87.52503 30.37249     1     3 alabama      <NA>
4 -87.53076 30.33239     1     4 alabama      <NA>
5 -87.57087 30.32665     1     5 alabama      <NA>
6 -87.58806 30.32665     1     6 alabama      <NA>
```

```
> plot_usmap(regions = "states") + theme(panel.background = element_blank())
> food_data <- food_data %>%mutate(County=tolower(County))
> food_data_updated <- left_join(states,food_data,by = c("region" = "County"))
Warning message:
In left_join(states, food_data, by = c(region = "County")) :
  Detected an unexpected many-to-many relationship between `x` and `y`.
i Row 352 of `x` matches multiple rows in `y`.
i Row 2 of `y` matches multiple rows in `x`.
i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to silence
  this warning.
> head(food_data_updated)
       long      lat group order  region subregion State FoodServices.97 FoodServices.2002
1 -87.46201 30.38968     1     1 alabama      <NA>                  6955               7075
2 -87.48493 30.37249     1     2 alabama      <NA>                  6955               7075
3 -87.52503 30.37249     1     3 alabama      <NA>                  6955               7075
4 -87.53076 30.33239     1     4 alabama      <NA>                  6955               7075
5 -87.57087 30.32665     1     5 alabama      <NA>                  6955               7075
6 -87.58806 30.32665     1     6 alabama      <NA>                  6955               7075
  FoodServices.2007
1              8093
2              8093
3              8093
4              8093
5              8093
6              8093
> ggplot(food_data_updated, aes(x=long, y=lat,group = group,fill = FoodServices.2007))+
+   geom_polygon(color = "yellow")
> coord_map("polyconic")
<ggproto object: Class CoordMap, Coord, gg>
    aspect: function
    backtransform_range: function
    clip: on
    default: FALSE
    distance: function
    is_free: function
    is_linear: function
    labels: function
    limits: list
    modify_scales: function
    orientation: NULL
    params: list
    projection: polyconic
    range: function
    render_axis_h: function
```

2) The Chicago_crashes.csv file contains information on every crash recorded in Chicago in June 2019 (see Chicago's portal at https://data.cityofchicago.org/Transportation/Traffic-CrashesCrashes/85ca-t3if for the latest data. I chose a random month because the data get dense quickly).

a. Create an appropriate type of geographic plot to show where all the accidents in this data occur.

Geographic Distribution of Traffic Accidents in Chicago - June 2019

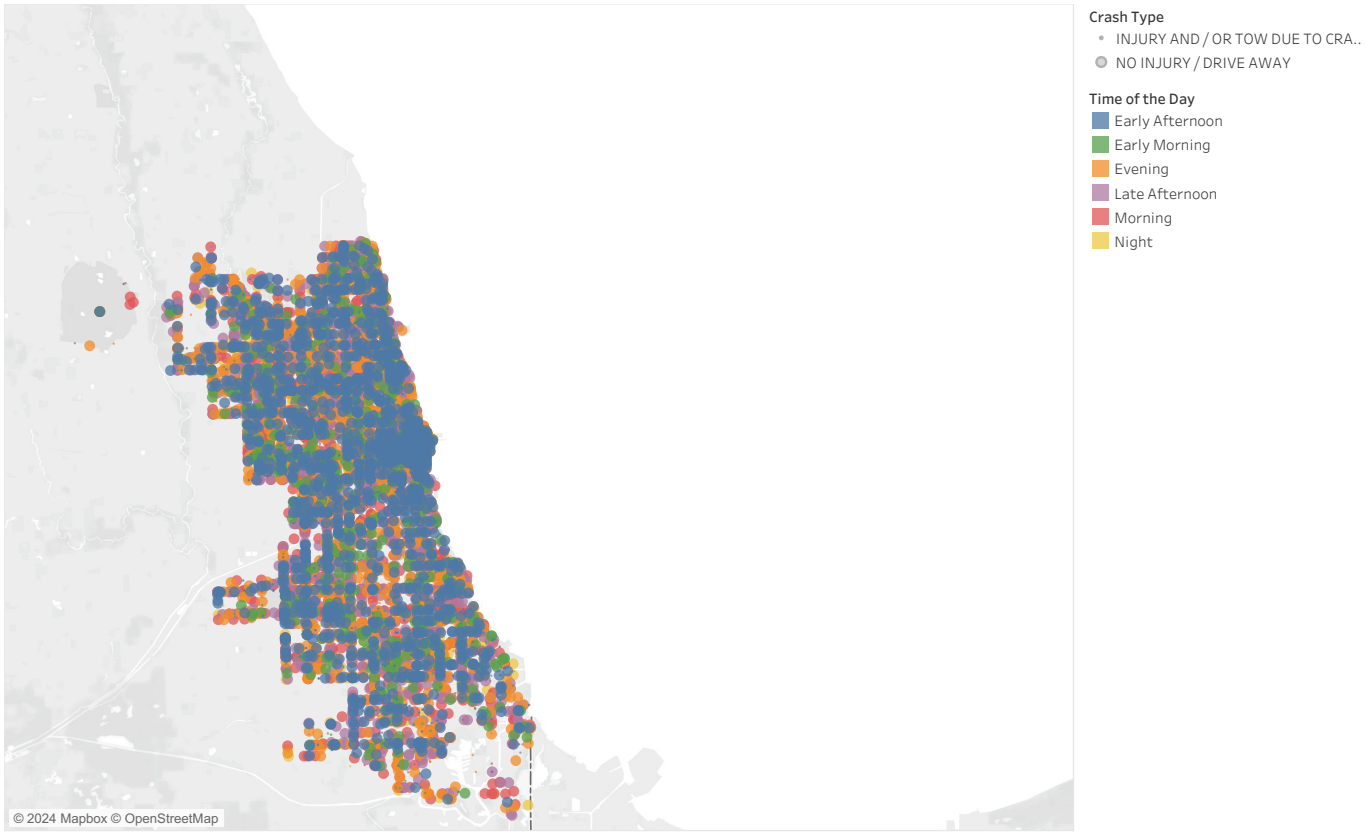


Map based on Longitude and Latitude. Color shows details about Street Direction.

This geographic plot illustrates the distribution of traffic accidents in Chicago for June 2019 based on street direction (N, S, E, W). The accidents are densely concentrated in the central areas of Chicago, particularly around downtown and nearby neighborhoods. The color-coding highlights the direction of the streets where the accidents occurred, showing a balanced spread across different street orientations. The map visually communicates where accidents are most common, with orange (E) and blue (W) directions dominating certain parts of the city, while red (S) accidents are concentrated in southern areas. This type of map helps to identify accident hotspots across different street directions in Chicago.

b. Create a visualization that shows how common crashes are in different parts of the city based on time of day. There are multiple approaches to this. Explain your approach and what you can see in your graph.

Crash Density by Time of the Day in Chicago



Map based on Longitude and Latitude. Color shows details about Time of the Day. Size shows details about Crash Type. The data is filtered on Crash Hour, which ranges from 0 to 23. The view is filtered on Time of the Day, which keeps 6 of 6 members.

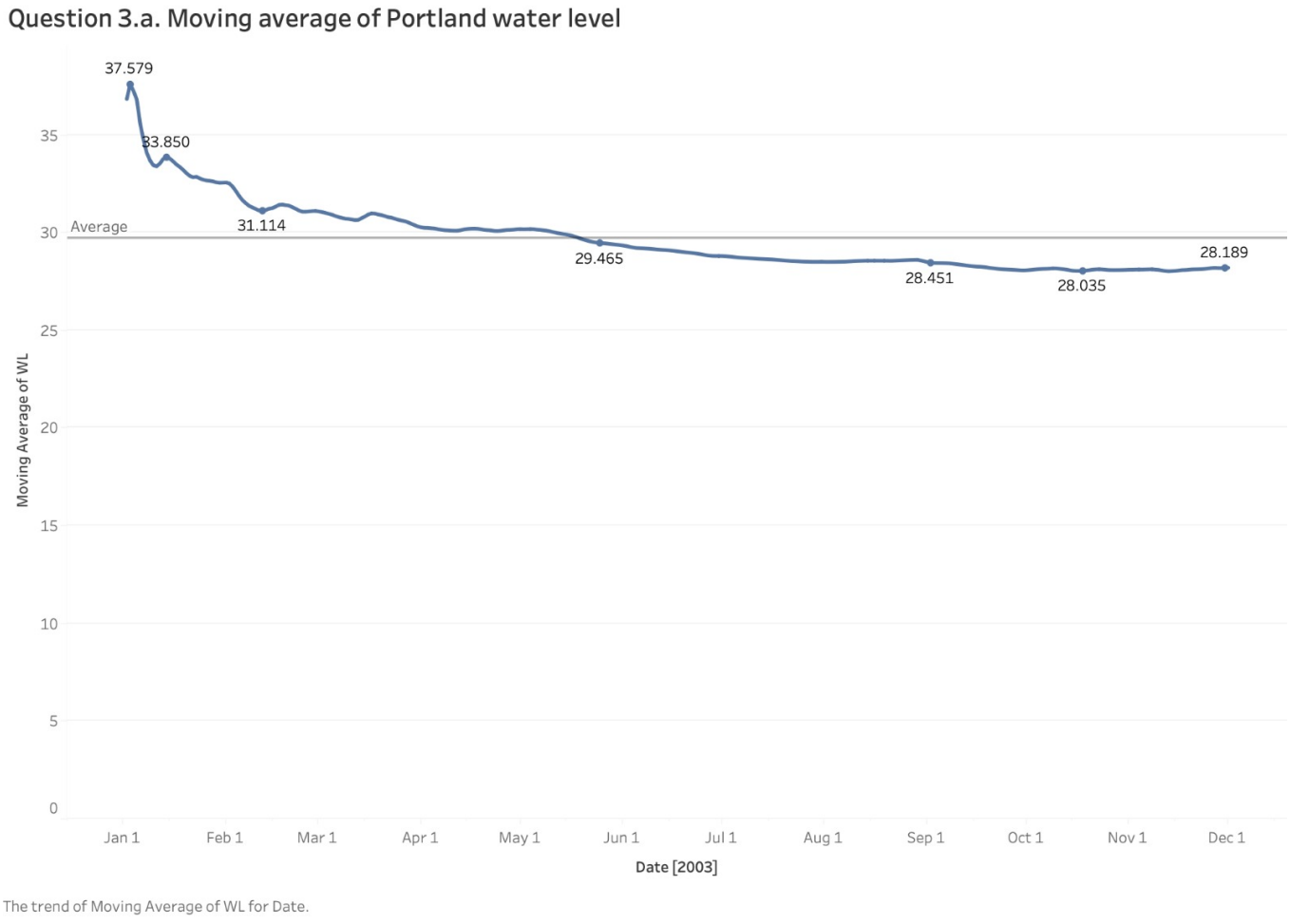
The visualization shows the density of crashes across Chicago based on the time of day. Each point on the map represents a crash, and the colors differentiate the time segments:

Morning crashes (yellow) and Evening crashes (red) appear to be more concentrated in central and southern areas of the city.
Early Afternoon (green) and Late Afternoon (pink) crashes also follow a similar pattern, though they seem more spread out across different regions.
Nighttime (purple) and Early Morning (blue) crashes tend to cluster in areas further south and closer to the lakefront.

This suggests that crashes are distributed across the city throughout the day but tend to concentrate in specific areas at different times. The map highlights that there is no one specific time period when crashes are most common but certain areas experience more crashes during the daytime compared to night.

3) (20 pts) Download the Portland Water Level dataset and explore it by creating the following visualizations of the time series from the techniques described in lecture. Use both R and Tableau for at least one question part. They should, of course, adhere to the design criteria that we've learned, and should clearly display the information described in each part.
a. This data contains a year of data with water level (WL) measurements every hour as a function of Time (i.e., 365 x 24 data points!). Since there is a lot of data, clean it up by

smoothing the data by calculating a moving average. Use a window approach with window size that covers a range of days (remember, the data is hourly) and graph the smoothed result. Work with the window to see what size window gives you the best view of the changes in the data while still smoothing the noise well.
Remember that the moving average is in the Quick-Table calculations inside of the right click menu on the data item in Tableau, and we can compute it in R quite easily as shown in the tutorial.



This graph shows the 7-day moving average of Portland water levels over the course of a year. The data, which consists of hourly measurements, has been smoothed using a moving average to reduce noise and reveal the overall trends in water level fluctuations.

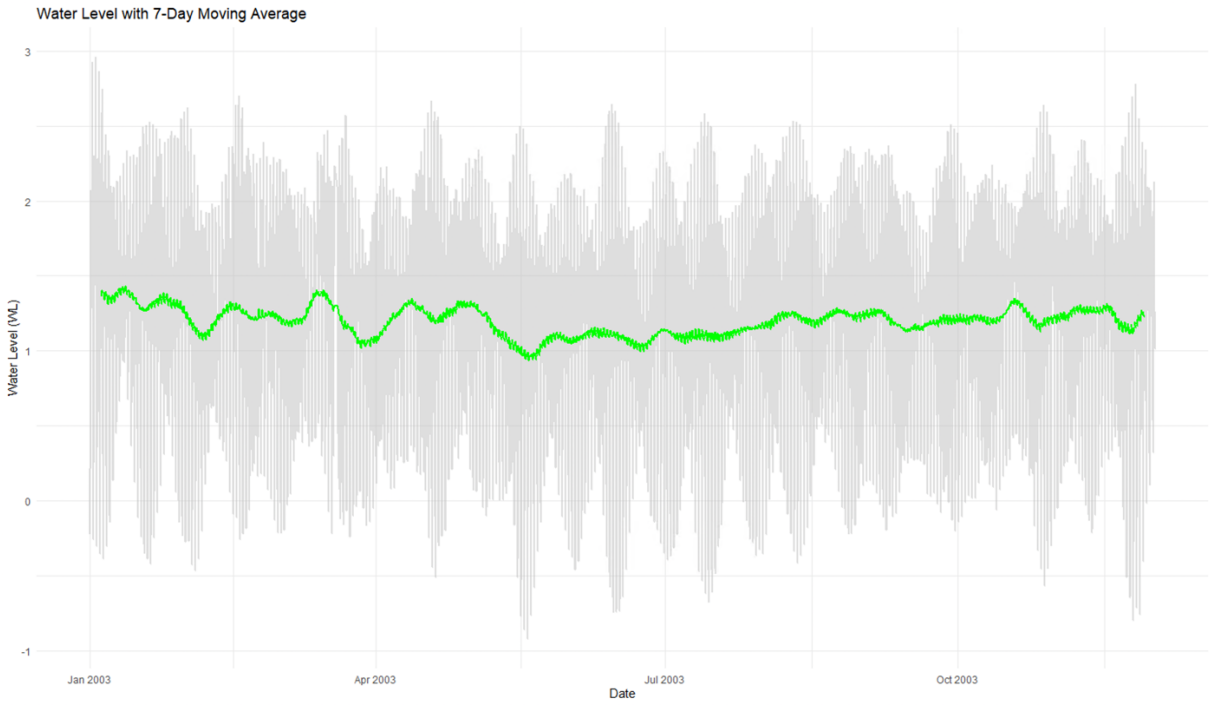
The water level starts at a higher point in January, peaking at around 37.6 early in the year. After this peak, the water level gradually declines and stabilizes around the 29-30 range by mid-year. There are some minor fluctuations throughout the year, but the general trend shows a slow decrease in the average water level from the start of the year through to the end of the year, where it reaches around 28.2.

Through R code-

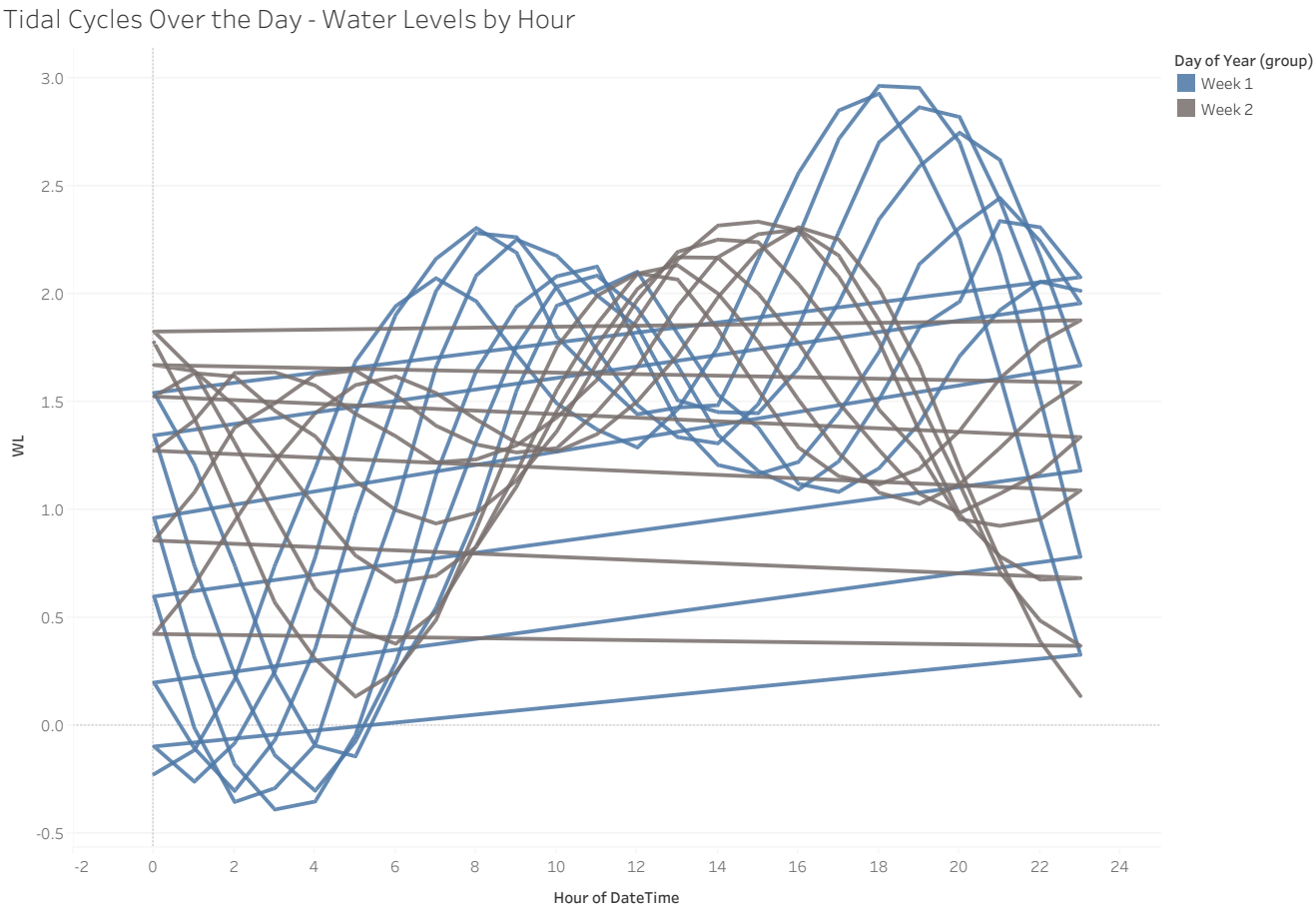
```
> rm(list = ls())
> library(dplyr)

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union

> library(ggplot2)
Need help? Try Stackoverflow: https://stackoverflow.com/tags/ggplot2
> library(zoo)

Attaching package: 'zoo'

The following objects are masked from 'package:base':

    as.Date, as.Date.numeric

> library(readr)
> data <- read.csv(file.choose(), header = TRUE)
> str(data)
'data.frame':   8040 obs. of  7 variables:
 $ Station: int  9431647 9431647 9431647 9431647 9431647 9431647 9431647 9431647 9431647 9431647 ...
 $ Date   : chr  "1/1/2003" "1/1/2003" "1/1/2003" "1/1/2003" ...
 $ Time   : chr  "0:00" "1:00" "2:00" "3:00" ...
 $ WL     : num  -0.226 -0.115 0.213 0.741 1.19 ...
 $ Sigma  : num  0.128 0.121 0.134 0.148 0.136 0.208 0.137 0.207 0.202 0.16 ...
 $ I      : int  0 0 0 0 0 0 0 0 0 0 ...
 $ L      : int  0 0 0 0 0 0 0 0 0 0 ...
> summary(data)
    Station            Date               Time                WL              Sigma        
 Min.   :9431647   Length:8040        Length:8040        Min.   :-0.926   Min.   :0.00000  
 1st Qu.:9431647   Class :character   Class :character   1st Qu.: 0.708   1st Qu.:0.03500  
 Median :9431647   Mode  :character   Mode  :character   Median : 1.274   Median :0.05400  
 Mean   :9431647                                        Mean   : 1.199   Mean   :0.06313  
 3rd Qu.:9431647                                        3rd Qu.: 1.725   3rd Qu.:0.08100  
 Max.   :9431647                                        Max.   : 2.962   Max.   :0.35100
```

```
       I                    L
 Min.   :0.000000   Min.   :0e+00
 1st Qu.:0.000000   1st Qu.:0e+00
 Median :0.000000   Median :0e+00
 Mean   :0.008707   Mean   :4e-04
 3rd Qu.:0.000000   3rd Qu.:0e+00
 Max.   :1.000000   Max.   :1e+00
                    NA's   :344
> # Check for missing values
> if (any(is.na(data))) {
+   cat("Warning: The dataset contains missing values. These will be removed.\n")
+   data <- na.omit(data)
+ }
Warning: The dataset contains missing values. These will be removed.
> # Convert Date and Time to POSIXct format
> data$DateTime <- tryCatch({
+   as.POSIXct(paste(data$Date, data$Time), format="%m/%d/%Y %H:%M")
+ }, error = function(e) {
+   stop("Error converting Date and Time: ", e$message)
+ })
> # Check for invalid DateTime values
> if (any(is.na(data$DateTime))) {
+   cat("Warning: Some DateTime values are invalid. These will be removed.\n")
+   data <- data[!is.na(data$DateTime), ]
+ }
Warning: Some DateTime values are invalid. These will be removed.
> # Sort data by DateTime
> data <- data[order(data$DateTime), ]
> # Calculate moving averages with different window sizes
> data$MA_24 <- rollmean(data$WL, k = 24, fill = NA)  # 1 day moving average
> data$MA_168 <- rollmean(data$WL, k = 168, fill = NA)  # 1 week moving average
> data$MA_720 <- rollmean(data$WL, k = 720, fill = NA)  # 1 month moving average
> # Create a plot with the original data and 7-day moving average (1 week)
> ggplot(data, aes(x = DateTime)) +
+   geom_line(aes(y = WL), color = "gray", alpha = 0.5, size = 0.8) +  # Original water level
+   geom_line(aes(y = MA_168), color = "green", size = 1) +  # 1 week (7-day) moving average
+   labs(title = "Water Level with 7-Day Moving Average",
+        x = "Date",
+        y = "Water Level (WL)") +
+   theme_minimal()
Warning messages:
1: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```

Plot:



b. Graph the cycles that happen each day (because of tides). You might try overlapping many days' data as separate overlapping time series, using a level plot, a horizon graph, etc. The point of this exercise is to try to come up with a way of showing the progression of the tides over some period of time that is rich and detailed and which shows the pattern, but which is still readable and which doesn't clutter the graph.



DateTime Hour vs. WL. Color shows details about Day of Year (group). The data is filtered on Day of Year, which ranges from 1 to 100. The view is filtered on sum of Day of Year and Day of Year (group). The sum of Day of Year filter ranges from 1 to 14. The Day of Year (group) filter excludes 22, 23 and Week 3.

This graph shows the daily tidal cycles over two weeks by plotting water levels at different hours of the day. The overlapping lines for Week 1 and Week 2 reveal the regular rise and fall of tides, with high and low tides occurring at similar times each day. The graph effectively captures the cyclic nature of tides while remaining clear and uncluttered, allowing easy comparison between days.

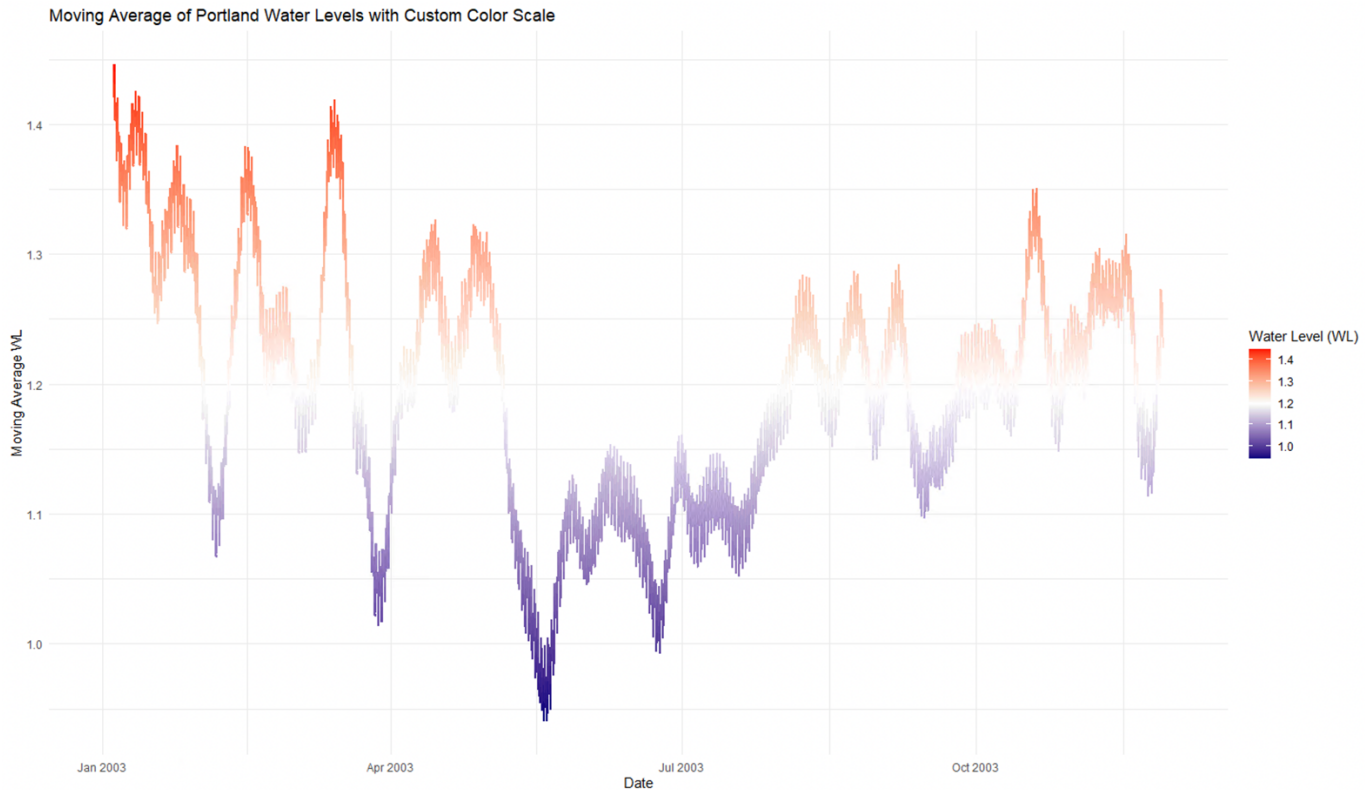
c. Then write a single paragraph outlining the differences between the information that each graph communicates.

In first graph (3.a), we can observe that the water level and its maximum value on a specific day in a particular hour of the month can be observed. This helps us to easily understand the peak tides and identify various water levels during different hours of the day.
The R code graph provides information per month. So, it illustrates information of the level of water for various months in a year.
In the second part's graph, the graph does seem similar, by carefully examining, one can observe that the overall variation is present in the data. The trend appears to be negative with a peak in certain periods and a low point in other, similar to an alternate approach.

4) Return to the Portland Water Level dataset. Recreate one of your plots from Question 3 with a custom color scale. Specifically, create a divergent color scale with the average water level at the midpoint and two separate colors used to show when the water is getting very high and very low. The point of this exercise is to experiment with creating a color scale, so choose your own distinctive colors to use for the endpoints and center. Make sure that they are reasonable choices given what you know about color scales. Use HSV space to choose the colors and explain how you made your decision. In Tutorial 4, you can see how to create color scale in ggplot that is interpolated in Lab space.

```
package 'scales' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
        C:\Users\local_SDHURVE\Temp\RtmpEB63wI\downloaded_packages
> library(ggplot2)
> library(scales)
> data <- read.csv(file.choose(),header = TRUE)
> str(data)
'data.frame':   8040 obs. of  7 variables:
 $ Station: int  9431647 9431647 9431647 9431647 9431647 9431647 9431647 9431647 9431647 9431647 ...
 $ Date   : chr  "1/1/2003" "1/1/2003" "1/1/2003" "1/1/2003" ...
 $ Time   : chr  "0:00" "1:00" "2:00" "3:00" ...
 $ WL     : num  -0.226 -0.115 0.213 0.741 1.19 ...
 $ Sigma  : num  0.128 0.121 0.134 0.148 0.136 0.208 0.137 0.207 0.202 0.16 ...
 $ I      : int  0 0 0 0 0 0 0 0 0 0 ...
 $ L      : int  0 0 0 0 0 0 0 0 0 0 ...
> data$DateTime <- as.POSIXct(paste(data$Date, data$Time), format="%m/%d/%Y %H:%M")
> data$MovingAvg_WL <- zoo::rollmean(data$WL, k = 168, fill = NA) # Example moving average
> # Calculate average water level
> avg_water_level <- mean(data$MovingAvg_WL, na.rm = TRUE)
> # Create the plot with a custom color scale
> ggplot(data, aes(x = DateTime, y = MovingAvg_WL)) +
+   geom_line(aes(color = MovingAvg_WL), size = 1) +
+   scale_color_gradient2(low = hsv(h = 240/360, s = 1, v = 0.5),  # Dark Blue for low values
+                         mid = hsv(h = 0, s = 0, v = 1),         # White at the midpoint (average)
+                         high = hsv(h = 0, s = 1, v = 1),        # Red for high values
+                         midpoint = avg_water_level,             # Midpoint set at average water level
+                         name = "Water Level (WL)") +            # Color scale label
+   labs(title = "Moving Average of Portland Water Levels with Custom Color Scale",
+        x = "Date",
+        y = "Moving Average WL") +
+   theme_minimal()
Warning messages:
1: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
This warning is displayed once every 8 hours.
Call `lifecycle::last_lifecycle_warnings()` to see where this warning was generated.
2: Removed 168 rows containing missing values or values outside the scale range (`geom_line()`).
```

Plot:



This graph shows the moving average of Portland water levels over the year 2003, using a custom divergent color scale to represent changes in water levels. High water levels are depicted in red, while low water levels are shown in blue/purple.
The white section represents the average water level, acting as the midpoint.
The water levels fluctuate throughout the year with several peaks (red areas) indicating periods of higher water levels, likely due to tides or seasonal factors, and troughs (blue areas) showing lower levels. The smooth transitions between colors highlight the gradual rise and fall of water levels, making it easy to identify trends and patterns over time.